

Basic Scale Document

Version 1 | August 2026

Current-state scalability analysis, known limitations, and practical growth paths for the existing product architecture.

1. Purpose

The purpose of this document is to describe how the Service Mobility Dashboard currently supports basic application scale, identify the main areas that may become bottlenecks as usage grows, and describe the improvements that would be required to support a significantly larger system.

The current product is designed primarily for an internal operational environment with a moderate number of service dispatchers and service tasks.

The system is not currently designed for internet-scale traffic or millions of users. However, its architecture already includes several decisions that reduce unnecessary processing and network traffic, while leaving clear paths for future optimization.

2. Expected Scale

The current version is designed to support an organization with:

- Dozens of active service vehicles.
- Dozens or hundreds of service tasks per workday.
- Multiple service dispatchers using the dashboard concurrently.
- Several days of future service-task data available for scheduling recommendations.

At this scale, the current architecture is expected to remain practical because most application operations are restricted to a specific day or a short date range.

If the system were expanded to support significantly larger fleets, thousands of daily tasks, many organizations, or a large number of concurrent dispatchers, several parts of the architecture would need to be improved.

3. Current Architecture and Scale Characteristics

The system is built using:

- Next.js for the application.
- Supabase/PostgreSQL for service-task storage and authentication.
- Vercel for application deployment.
- Mapbox for map rendering, geocoding, and route optimization.

The architecture separates responsibilities between the server and the browser.

Server responsibilities

- Authentication verification.
- Loading service-task information from Supabase.
- Retrieving recommendation candidates.
- Protecting access to organizational data.

Client responsibilities

- Interactive map behavior.
- Grouping and presenting vehicles.
- UI filtering and selection.
- Map rendering.
- Route optimization requests to Mapbox.
- Local interactive state.

This separation prevents ordinary map interactions such as selecting a task, focusing on a vehicle, or hiding a vehicle from creating additional database requests.

4. Daily Service-Task Query

The main dashboard retrieves service tasks using the selected `scheduled_date`. Instead of loading the organization's entire service-task history, the application queries only the tasks required for the selected workday.

```
SELECT *
FROM service_tasks
WHERE scheduled_date = selected_date
ORDER BY vehicle_id, time_window;
```

Current scale advantage

For the normal dispatcher workflow, the relevant dataset is naturally bounded by one workday. A dispatcher viewing August 20 does not need tasks from August 19, August 21, or historical service data. This provides a form of application-level data partitioning by date.

Potential bottleneck

As the `service_tasks` table grows to contain a very large history of service records, filtering and sorting by `scheduled_date`, `vehicle_id`, and `time_window` may become more expensive if the database is required to scan a large portion of the table.

Future improvement

A production system with a large service-task history should use an index supporting the daily query. For example:

```
CREATE INDEX idx_service_tasks_daily_schedule
ON service_tasks (scheduled_date, vehicle_id, time_window);
```

The current project repository does not explicitly define this index, so this is considered a future scale improvement rather than an already implemented feature. The actual index design should be validated against production query plans and real workload before deployment at larger scale.

5. Recommendation Query

When the dispatcher searches for the location of a new service request, the application examines service tasks scheduled during the upcoming four days.

1. Retrieves tasks inside the four-day date window.
2. Removes tasks with invalid coordinates.
3. Calculates the geographical distance between the new location and each candidate task.
4. Keeps only tasks within a 20 km radius.
5. Keeps the closest task for each vehicle/date combination.
6. Sorts the final recommendations by distance.

Current scale advantage

The recommendation query does not examine the complete database. It is limited to a four-day operational window, which keeps the working dataset relatively small for the expected project scale.

Potential bottleneck

The geographical filtering itself currently happens in application logic after the records are retrieved from Supabase. If the organization eventually has thousands or tens of thousands of tasks scheduled within four

days, the application would transfer many unnecessary rows, perform a Haversine distance calculation for every candidate, and discard most records only after they reach the application server.

Future improvement

For a much larger fleet, geographical filtering should move closer to the database. Possible solutions include:

- PostgreSQL/PostGIS spatial queries.
- A geographical index.
- An initial latitude/longitude bounding-box filter before calculating exact distance.

This would reduce database-to-server data transfer, server-side calculations, and response latency.

6. Route Optimization and Mapbox Requests

For each vehicle containing at least two valid service locations, the client requests an optimized route from the Mapbox Optimization API. Vehicles are processed independently, so a slow or failed route calculation for one vehicle does not prevent other vehicle routes from being calculated.

Avoiding unnecessary requests

Memoized route inputs

Route requests are generated from the actual set of vehicle coordinates. Normal UI interactions such as hovering over a marker, selecting a task, focusing on a vehicle, or hiding a vehicle do not create a new optimization request if the underlying route coordinates have not changed.

Client-side route cache

Successful and failed optimization results are stored in an in-memory client cache using the combination of vehicle ID and route coordinates. If the same route is requested again during the current client lifetime, the existing result can be reused.

Parallel independent requests

Routes for different vehicles are requested independently rather than sequentially. One slow vehicle therefore does not block the rendering of every other route.

Request cancellation

If a route request is no longer relevant because its inputs have changed, the application can cancel the in-progress request, preventing obsolete network work from continuing unnecessarily.

7. Mapbox Optimization Limit

The current Mapbox Optimization API used by the application has a maximum of 12 coordinates per optimization request. The application explicitly handles this limitation.

- The first 12 valid tasks participate in route optimization.
- Additional valid tasks remain visible as map markers.
- The application does not attempt to send an invalid oversized request.

Current limitation

A large service route with more than 12 stops is not fully optimized by the current system. This is an intentional limitation of the current version rather than silently sending unsupported requests.

Future improvement

- Dividing a large route into geographical groups.
- Performing multiple optimization requests.
- Using a routing service that supports larger vehicle-routing problems.
- Implementing a dedicated Vehicle Routing Problem optimization service.

The appropriate approach would depend on fleet size and business requirements.

8. External API Scale

The dashboard relies on Mapbox for base map rendering, geocoding, and route optimization. The application server does not perform route optimization itself. This reduces computational load on the Next.js backend, but external API usage increases as the number of dispatchers and vehicles increases.

Current limitation

The route cache currently exists only in the individual browser session. It is not a shared organizational cache. Therefore, two different dispatchers looking at the same vehicle route may still generate separate Mapbox requests.

Future improvement

At larger scale, repeated route calculations could be reduced by introducing a shared cache. A route result could be cached according to date, vehicle ID, and the ordered set of service-task coordinates.

- A server-side cache.
- Redis or another distributed cache.
- A dedicated database table containing calculated daily routes.

The application would first check whether a valid route already exists before requesting another Mapbox calculation. This could significantly reduce external API requests, API cost, and latency.

9. Avoiding Unnecessary Data Loading

Date-scoped database requests

Only tasks for the requested workday are loaded into the main dashboard.

Short recommendation window

Recommendation searches inspect only four upcoming days rather than the complete task history.

Client-side interaction

Once the day's tasks are loaded, vehicle visibility, task selection, vehicle focus, map bounds calculations, and task grouping are performed locally without repeatedly querying Supabase.

Memoization

Derived values such as vehicle groups, route request inputs, and map bounds are memoized so that unrelated React re-renders do not repeatedly perform the same calculations.

10. Pagination

The current dashboard does not use traditional pagination for daily service tasks. This is intentional for the current product.

The main purpose of the dashboard is to provide the dispatcher with a complete geographical view of a workday. Loading only one page of tasks would produce an incomplete operational map and could hide important vehicle activity.

Because the database query is already limited to one selected workday, the expected number of records remains manageable at the current scale.

When pagination would become necessary

If a future organization has thousands of tasks per day, loading every record into the sidebar and rendering every marker at once may become inefficient. Possible improvements include:

- Virtualizing long task lists.
- Paginating tabular views that do not require the complete map dataset.
- Loading map records according to the visible geographical viewport.
- Server-side aggregation.
- Marker clustering.

Pagination is therefore not currently required for the primary map workflow, but alternative data-loading strategies would be required if daily datasets grow substantially.

11. Client-Side Map Rendering

Every valid service location is currently represented by an interactive map marker. This works well for the expected number of daily service tasks.

Potential bottleneck

If a single workday contains hundreds or thousands of visible markers, browser performance may begin to degrade because the client must manage large numbers of React marker components, user interaction handlers, route geometries, tooltip state, and map rendering.

Future improvement

- Marker clustering.
- GeoJSON source layers rather than individual React markers.
- Viewport-based loading.
- Progressive rendering.
- Aggregated operational views at lower zoom levels.

These techniques would reduce the number of interactive objects rendered simultaneously.

12. Server and Client Separation

The current application intentionally separates server-side data operations from client-side interactive operations.

Server-side responsibilities

- Authentication verification.
- Supabase database access.
- Daily task retrieval.
- Recommendation retrieval.

This prevents sensitive database access from being directly exposed as arbitrary client logic.

Client-side responsibilities

- Map rendering.
- Vehicle grouping.

- Marker interaction.
- Visibility controls.
- Route visualization.
- Route optimization requests.
- Local UI state.

Scale benefit

Common dispatcher interactions do not cause repeated Vercel/Supabase round trips. For example, hiding a vehicle from the map is a presentation decision and is handled locally rather than asking the server to reload the day's tasks.

13. Database Payload Size

The current daily query uses `select(*)` for the selected workday. This is acceptable for the current product because the dashboard uses many of the service-task fields when displaying operational information.

At a larger scale it may become beneficial to examine whether every database column is required for every view.

Future improvement

```
SELECT
  id,
  address,
  lat,
  lng,
  vehicle_id,
  installer_name,
  car_plate,
  customer_name,
  customer_phone,
  time_window,
  short_note,
  scheduled_date
FROM service_tasks
WHERE scheduled_date = ...;
```

This would reduce data transfer if additional large or unused SAP fields are added to the mirror in the future.

14. Concurrent Users

With dozens or hundreds of dispatchers, several users may request the same workday simultaneously. Each dashboard load currently requires authentication verification, one date-scoped Supabase task query, and client-side route requests for relevant vehicles.

For the expected internal usage, this structure is relatively lightweight. As concurrency increases, however, database query frequency, external Mapbox usage, authentication traffic, and duplicate requests for identical daily data may also increase.

Future improvement

- Shared caching of frequently requested daily data where compatible with authentication requirements.
- Shared route caching.
- Database connection and query monitoring.
- Rate monitoring for external APIs.

- Query-performance monitoring.

The application should be optimized based on measured production bottlenecks rather than assuming every layer requires caching in advance.

15. Main Current Scale Limitations

1. Recommendation calculations are performed in application logic: all tasks from the four-day range are retrieved before geographical filtering.
2. No database index is explicitly defined in the project repository for the main daily query; as the historical table grows, indexing by schedule date becomes increasingly important.
3. Route caching is local to each browser, so different dispatchers may calculate the same Mapbox route independently.
4. Route optimization is limited to 12 coordinates per vehicle.
5. All map markers for the selected day are rendered simultaneously.
6. The daily task query currently retrieves all columns.
7. There is no shared application-level cache for daily operational data.

These limitations are acceptable for the expected initial scale but should be addressed before supporting much larger fleets or significantly higher traffic.

16. Future Scale Improvements

- 1. Database indexing:** Add and verify indexes for the most common database access patterns, especially queries based on `scheduled_date`.
- 2. Database-level geographic filtering:** Move recommendation candidate filtering closer to PostgreSQL using spatial indexing, PostGIS, or bounding-box queries.
- 3. Shared route caching:** Cache Mapbox optimization results so multiple dispatchers do not repeatedly calculate identical vehicle routes.
- 4. Large map dataset optimization:** Introduce marker clustering, GeoJSON layers, virtualization, or viewport-based loading.
- 5. Advanced route optimization:** Replace or extend the current 12-stop optimization approach for vehicles with larger daily schedules.
- 6. Monitoring:** Introduce production monitoring for database query latency, number of records scanned, Mapbox API usage, failed external requests, server response times, and client rendering performance.
- 7. Load testing:** Perform load tests using realistic dispatcher behavior before increasing the expected production user count.

17. Summary

The current Service Mobility Dashboard is designed for a moderate internal operational workload rather than internet-scale traffic.

Several architectural decisions already support basic scale:

- Database queries are limited by workday.
- Recommendation searches use a short four-day window.
- Interactive filtering and grouping happen on the client.
- Derived UI data is memoized.
- Mapbox route requests are cached within the client session.
- Different vehicle routes are processed independently.
- Obsolete route requests can be cancelled.
- The system explicitly handles Mapbox's route-size limitation.

For the expected initial workload of dozens or hundreds of users and a moderate daily fleet, this architecture provides a reasonable balance between simplicity and performance.

The primary future scale improvements would involve database indexing, database-level geographical filtering, shared route caching, more efficient rendering of very large map datasets, and more advanced routing support.

The current approach intentionally avoids introducing distributed infrastructure before it is required, while keeping clear upgrade paths for future growth.